



tecnun

CAMPUS TECNOLÓGICO DE LA UNIVERSIDAD DE NAVARRA
NAFARROAKO UNIBERTSITATEKO CAMPUS TEKNOLOGIKOA
Escuela Superior de Ingenieros · Ingeniarien Goi Mailako Eskola

● C++ Laboratory Lessons

Lesson N° 5



C++
PARA INGENIEROS SUPERIORES

Computer Programming II

Prof. Dr. Paul Bustamante

OUTLINE

1	
OUTLINE.....	1
1. Introduction.....	1
1.1 Exercise 1: My First Function.....	1
1.2 Exercise 2: A program for primes.....	1
1.3 Exercise 3: Calculating the Roots of a Function.....	1
1.4 Exercise 4: Series representation of e^x	2

1. Introduction.

As we have been doing so far, the first exercise of this lesson should be done alone, so you can gain more experience in programming in C++.

1.1 Exercise 1: My First Function

This exercise consists of making a program that calculates the distance between two points (x, y) given by the user. You must create the following function:

```
double Distance( double x1, double y1, double x2, double y2);
```

In order to write the code, you must create a Project **Exer1** and the file **FirstFunc.cpp**.

1.2 Exercise 2: A program for primes

The next exercise consists of writing a program that obtains all the prime numbers between two numbers A and B given by the user. Remember, **you must create a function called Permute** that allows to swap the values of A and B only if A is not smaller than B.

In case you do not remember it, a prime number is a number that can only be divided by the number itself and by one. You can use the *operator module* (%) for this algorithm.

Besides the function **Permute**, you must create the function **Prime** that takes one argument: an integer containing the number to be identified as prime or not. The function must return a Boolean value (true or false). The function's prototype can be the following:

```
bool Prime( int A);
```

You must create a Project **Exer2** and the file **Prime.cpp** for the code.

1.3 Exercise 3: Calculating the Roots of a Function

This exercise consists of making a program that finds the roots of a function between two given points (a, b) using the midpoint method. The algorithm consists of making successive iterations, dividing the interval into two and choosing a new interval in which the root is (check if $(f(a) \cdot f(b) < 0)$). The method ends when the root is near a value ($ZERO = 1e-9$) or upon reaching a maximum number of iterations the root has not been found.

You must create a project called **Exer3** and the file **roots.cpp**, which will contain the following code:

```

//PB: obtaining the roots of a function using the midpoint method
#include <iostream>
using namespace std;
#include <math.h>

double fx( double);
double Midpoint( double a, double b, int max, int &num);
void main()
{
    double a,b;
    int num,max=300;

    cout << "Enter the interval to be evaluated, a b:";
    cin >> a >> b;
    //add the interval checking statements

    double root = Midpoint(a,b,max,num);
    //Print the results on the screen

}
double Midpoint( double a, double b, int max, int &num)
{
    double m, ym, ZERO=1e-9;
    num=0; //important
    //write the root calculating algorithm using the midpoint method
    return m;
}
double fx( double x) //x in radians
{
    double f = x - cos(x); //roots between 0 and PI
    return f;
}

```

1.4 Exercise 4: Series representation of e^x

In this exercise, you must find the value of the function e^x using two criteria to stop the algorithm. You must create a function for each one:

1. Number of iterations: `double Exp( double x, int Num);`
2. Amount of error between an iteration and the next one:

`double Exp( double x, double Error, int &Num);`

The series representation of e^x in:

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

Remember that in the second case, the function must return the number of iterations employed in the calculation, and afterwards, it has to be printed on the screen.

You can use the function `exp(x)`, defined in *math.h*, to check if the result obtained is correct or not.

You must create a project called *Exer4* and the file *FuncExp.cpp* to write the code.